

FRONTEND DESIGN — what *LBM Solver Studio* must carry for the WB HOME4 D3Q27 hydro + phase-field stack, and what would set it apart

Design note for the frontend build

HOME4 project note

2026-10-05

Abstract

The first prototype of *LBM Solver Studio* is a clean, well-built shell whose *shape* was borrowed from finite-volume Navier–Stokes products (Fluent, SU2, STAR): a meshing step, material properties, a solver page, residual-style monitors, and a post-processor. HOME4 has none of the first three in that form and offers several things an FV code cannot. This note gives the frontend engineer the dozen facts about the solver that should shape the interface, a specific critique of the prototype, a replacement information architecture, and inventories — fields, monitors, forces, zones, multidomain, run/backends, outputs, viewer — each mapped to the solver attribute or driver flag that supplies it, marked **exists** (the code already produces it) or **add** (a small solver-side addition is needed). It closes with the ranked list of features that would make the product different from every other CFD front end, a phased build sequence, a proposed telemetry schema so the GUI never has to import **torch**, and a worked map of the `run_hull_speed.py` flags onto controls. The thesis in one sentence: **organise the interface around lattice units, ledgers and budgets, the diffuse interface, and validated recipes — not around meshing, residuals and materials.**

Contents

1	Scope	2
2	Twelve facts about the solver that should shape the interface	2
3	The prototype: keep, change, remove	3
4	Information architecture	4
5	Setup: dimensionless first, with a live feasibility map	5
6	Field inventory	5
7	Monitors: ledgers and budgets before anything else	6
8	Forces, bodies, attitude	7
9	Boundaries and zones	8
10	Multidomain	8
11	Run: backends, cost, queue, reproducibility	8
12	Outputs and the timeline	8

13 Viewer: adopt the kinetic_viz contract	9
14 Validation cards and recipes	9
15 The trouble locator	10
16 What sets it apart — ranked	10
17 Build sequence	11
18 Telemetry schema (so the GUI never imports torch)	11
19 Open questions for Colton and the frontend engineer	11
A Worked map: run_hull_speed.py flags → controls	12

1 Scope

This note covers the 3-D stack in `home4_hydro_phase_3d`: the single-domain velocity-based D3Q27 solver (`home4_3d_multiphase.py`), the well-balanced layer (`home4_3d_wb.py`), the FSI and moving-body layers (`home4_3d_fsi_solver.py`, `home4_3d_moving_fsi.py`, `hull_moving.py`), the multidomain stack (`home4_3d_md_*.py`, `multidomain_nlevel.py`), the CAD→SDF pipeline (`cad_sdf.py`), the run drivers (`run_*.py`), the PyVista viewer (`kinetic_viz/`) and the rack workflow (`rack/`, `RACK_HANDOFF.md`). The mathematical content is in `home4_3d_hydro_phase_math.tex`; equation references below point there. The 2-D stack is out of scope but the same architecture applies.

The note is written for someone who will build the interface without having read the solver. Where it says “the solver stores X ” it names the attribute. Where it proposes a new diagnostic it says what has to be added and roughly where.

2 Twelve facts about the solver that should shape the interface

1. **Everything runs in lattice units.** Lengths in cells, time in steps, speed in cells/step with $c_s = 1/\sqrt{3}$. Physical units are a *map* chosen at setup (Δx , Δt , ρ_{ref}), not a property of the fields. The user, however, thinks in Re, Fr, Bo, We, Ca, Pe, At, KC, Ga; every benchmark is specified and reported that way (math note §2.2).
2. **There is no mesh.** There is a Cartesian lattice ($N_x \times N_y \times N_z$), a signed-distance band around CAD bodies (`cad_sdf.py`: tessellate, weld, orient, band distance, winding-number sign, EDT completion), cut links with interpolation weights, and multidomain refinement patches with factor-2 levels. “Meshing” as a stage does not exist; “Lattice” and “Geometry” do.
3. **The scheme is explicit and time-accurate.** There are no iterations within a step, hence no residual to converge. The natural health signals are conservation ledgers, budget closures and agreement between independent channels (§7).
4. **The zeroth moment is pressure, not density.** The hydro population g carries p^* (normalised pressure); ρ is a function of ϕ (`update_density_from_phi`). Under the WB layer p^* is the dynamic pressure p_d and the hydrostatic head Π_h is carried separately; physical pressure is reassembled as $p = \rho c_s^2 p_d + (\Pi_h - \Pi_h^0)$ (`WBFSI3D.measure_forces`).
5. **The strain rate is a state variable.** S_{ab} (six components) is stored and relaxed in closed form; u , p^* , S and the seven tied third-order closures $a^{(3)}$ are the hydro state. Dissipation $2\nu S:S$, enstrophy and Q come without finite differencing.
6. **Relaxation time is a field.** $\tau(\phi)$ (`tau fld`) runs between τ_H and τ_L ; the light phase is the dangerous end (ν_L small, $\tau_L \rightarrow \frac{1}{2}$).
7. **The interface is a diffuse object with its own parameters.** ξ in cells (4–5), σ , mobility M , four phase relaxation rates ($s_T, s_{D1}, s_{D2}, s_{xy}$), the `muphi` force form, and two load-bearing

safeguards (gradient limiter at $1.6\times$ the tanh slope; force thresholding at $60|F_b|$ with factor 0.6 where $\phi < 0.1$; math note §5.3).

8. **Forces are a decomposition.** $F = F_s + F_p + F_\nu + F_b$ (single domain) or $F_s + F_p(p_d) + F_\nu + F_{\text{tilt}}$ (WB), each a $[3, N_x, N_y, N_z]$ field the user may want to see separately.
9. **Bodies are SDF bands, and moving them has a cost cliff.** Static mask: 47.8 MLUPS; per-step retabulation: 5.9 (oscillating cylinder) and 0.63 (sedimentation) on the M1 Pro (math note, performance summary table). Two independent force channels exist: stress integral and momentum exchange.
10. **Multidomain has per-level physics.** With $k = 2^d$: $\nu^{(d)} = \nu_0 k$ (or a τ floor), $\sigma^{(d)} = \sigma_0 k$, $M^{(d)} = M_0 k$, $g^{(d)} = g_0/k$; ξ is fixed in *root* cells; mobility is a fine-level quantity; each level keeps a mass ledger (math note §10.5, Prop. 10.1).
11. **Three backends, one of them silently slow.** Metal (Mac), CUDA (rack), or the pure-PyTorch fallback when the extension fails to import — tens of times slower with no other symptom than a stdout line. The code is bandwidth-bound (~ 500 B/node/step for two-buffer D3Q27).
12. **Four kinds of output already exist.** Traces (every N steps), mid-span slices, full 3-D viz snapshots (`kinetic_viz.data.save_snapshot`: `ux`, `uy`, `uz`, `phi`, `solid`, `iteration`, `spacing`, `origin`; atomic rename), and restart checkpoints (`save_state`: the full moment state incl. S , $a^{(3)}$, J_ϕ , P_ϕ). They are different things and belong on the timeline as different markers.

Table 1: Translation table for the frontend engineer: the FV concept the prototype was modelled on, and the HOME4 counterpart.

FV-GUI concept	HOME4 counterpart
Mesh / meshing	Lattice resolution + unit map; SDF band from CAD; cut links; MD levels
Material properties	Two fluids ($\rho_H, \rho_L, \nu_H, \nu_L$) and an interface (σ, ξ, M, s^ϕ)
Residual convergence	Ledgers (mass, energy), channel agreement (forces), rest tests (WB)
CFL number	Lattice Mach U/c_s and τ margin; interface Pe and Cn
Pressure field	p^* , p_d , Π_h , reassembled p — four layers, not one
Velocity gradient (post-processed)	S_{ab} stored; dissipation/enstrophy/ Q exact from state
Turbulence model panel	None; instead the kinetic layer (S^{neq} , $a^{(3)}$, filtered $a^{(4)}$)
Inlet/outlet/wall BCs	Plus sponges, beaches, wave absorption, phase-wall pinning, mass correction
Time step size	Fixed by the lattice; the knobs are N , U (Ma) and τ
Checkpoint	Viz snapshot <i>or</i> restart state; not the same file

3 The prototype: keep, change, remove

Keep. The visual language (dark, quiet, dockable panels with collapse buttons), the top bar (Run / Pause / Stop / Step / Export), the playback bar with a snapshot index, the axes gizmo, the colour-scale panel with a “source range” line, the view presets (Flow overview, Perspective), the notification bell, and the recovery banner. *Step* is especially right for an LBM: one step is cheap and well-defined; add “step N ” and “run to $t^* = \dots$ ”.

Change.

- The colour bar reads 0.005–397 m/s for an airfoil. At lattice Mach above ~ 0.3 this solver diverges; 397 m/s is supersonic in air. The demo numbers show the shell has no notion of the unit map. Every scalar readout needs lattice, physical and nondimensional forms, with one global toggle and the other two in a tooltip.
- The playback bar shows “frame 600, 0.1200 s”. Show step, physical time, and nondimensional time (t/T , tU/L , t/t_0) together; the last is the one every validation plot is drawn against.

- The demo project is “Airfoil SU2 009” (single-phase, compressible, FV). The shipped example projects should be the validated recipes (§14).
- “Meshing” → *Lattice*. “Materials” → *Fluids & Interface*. “Boundary Conditions” → *Boundaries & Zones*. “Solver” → *Run*. “Results” and “Post-Processing” merge into *Fields* and *Reports*; add *Bodies* and *Validation*.
- The status line “RECORDED CFD” is honest and should stay as a mode badge: *live*, *replay*, *remote* (job on the rack).

Remove. Nothing visual. Remove the assumption that a project is one geometry + one mesh + one solver run; a HOME4 project is a *recipe lineage* (§14) with many runs, each a provenance `.npz`.

4 Information architecture

Sidebar, top to bottom, and what each page owns:

Dashboard

Open projects, running jobs (local and remote), last ledgers, last failures with their locator (§15).

Projects

Recipe lineage tree; each run as a card with its `.npz`, tag, backend, MLUPS, and gate status.

Geometry

CAD import (IGES/STEP/STL/OBJ via `cad_sdf.py`), attitude (heel/trim/yaw/sinkage), flotation solve (`--float`), SDF band preview against the voxel staircase, cut-link fraction, tessellation cache status.

Lattice

N_x, N_y, N_z ; unit map; pads/air gap/depth in body lengths (`--pad_up`, `--pad_down`, `--pad_side`, `--depth`, `--air`); MD levels with per-level parameters; memory and cost estimate.

Fluids & Interface

$\rho_H, \rho_L, \nu_H, \nu_L, \sigma, \xi, M, s^\phi$, `tau_method`, `surface_tension_form`, safeguards; the dimensionless-first form (§5) lives here.

Boundaries & Zones

Walls, inlet/outlet with ramp, periodic axes, sponges and beaches, wave absorption, phase-wall pinning ($\phi_{\text{top}}, \phi_{\text{bot}}$), mass-correction mode, drawn as translucent regions in the 3-D view.

Bodies

Fixed / forced (heave, pitch, spin) / free; mass properties, CoG, stiffness matrix K ; retabulation policy and its cost; force channels.

Run

Backend, device, block shape, output cadence (`--measure_every`, `--print_every`, `--viz_every`, `--save_state`), warm start (`--init_state`), tag, outdir, queue target (local / gpulab), live CLI equivalent.

Monitors

Ledgers and budgets first; forces; performance; safeguard activity.

Fields

The 3-D viewer (§13).

Validation

Recipe cards with reference overlays and gates (§14); convergence-ladder builder.

Reports

Figures with sidecars, tables, the run’s CLI line and parameter block; export to the LaTeX report conventions.

Settings

Unit display, palette, viewer defaults.

5 Setup: dimensionless first, with a live feasibility map

The user enters the dimensionless groups of the case and the lattice resolution; the GUI derives the lattice parameters and checks them. The relations are those already used in the drivers (`run_hull_speed.py` prints exactly this block at startup: U in lu/step, τ , Re_L , U/c_s , Fr_h):

$$\nu = c_s^2 \left(\tau - \frac{1}{2} \right), \quad \text{Ma} = U/c_s, \quad \text{Re} = \frac{UL}{\nu} = \frac{\text{Ma} L}{c_s \left(\tau - \frac{1}{2} \right)}, \quad (1)$$

$$\text{Kn} \approx \frac{\text{Ma}}{\text{Re}} = \frac{c_s \left(\tau - \frac{1}{2} \right)}{L}, \quad \text{Cn} = \frac{\xi}{L}, \quad \text{Pe}_\phi = \frac{UL}{M}, \quad (2)$$

$$\text{Fr} = \frac{U}{\sqrt{gL}}, \quad \text{Bo} = \frac{\rho_H g L^2}{\sigma}, \quad \text{We} = \frac{\rho_H U^2 L}{\sigma}, \quad \text{At} = \frac{\rho_H - \rho_L}{\rho_H + \rho_L}. \quad (3)$$

For fixed Ma and τ , $\text{Re} \propto L$: resolution buys Reynolds number. The three-way trade (N , Ma , τ) is the central fact of LBM sizing and should be a *live widget*: drag any one, the others respond, and the validated recipes are plotted as points so the safe region is visible. The widget replaces the Knudsen number as a display: Kn is derived from two things the user already controls, is tiny by construction in the hydrodynamic regime the Chapman–Enskog analysis assumes (math note §6), and is best shown as a tooltip.

Table 2: Setup checks the GUI should run before launch. All derive from parameters already in the drivers.

Check	Rule	Message / action
Lattice Mach	$U/c_s \leq 0.1$ (warn), ≤ 0.3 (block)	“Compressibility error $\sim \text{Ma}^2$ ”; suggest raising N or lowering U
τ margin	$\tau_L - \frac{1}{2} \geq 0.01$ (warn below)	Light phase under-resolved viscously; suggest <code>tau_method</code> , ν_L , or MD τ floor
Interface width	$4 \leq \xi \leq 5$ cells	Below: spurious currents and limiter firing; above: Cn too large
Cahn number	$\text{Cn} = \xi/L$ vs the recipe’s	Fixed- Cn refinement means $\xi \propto N$; warn when a ladder holds ξ in cells
Mobility	M matches recipe at the <i>finest</i> level	MD: pass $M_0 = M_{\text{ref}}/2^{d_{\text{max}}}$ (defect D23)
Density ratio	$\rho_H/\rho_L \geq 100$ flags safeguards	Turn on gradient limiter and force thresholding (both required at 1000:1)
Memory	≈ 350 B/node $\times N_{\text{cell}}$ (+ MD, FSI)	Compute from the allocation list, not a constant; M4 Pro MPS ceiling 79.5 Mcells
Index width	Metal path: $27N_{\text{cell}} < 2^{32}$ ($N_{\text{cell}} < 159$ M)	Block on Metal; CUDA path uses <code>size_t</code>
Backend	extension imported?	Big badge: Metal / CUDA / <i>PyTorch fallback (slow)</i>
Sponge sizing	width vs expected wavelength $2\pi\text{Fr}^2 L$	Hull driver already compares the wake wavelength to $2\pi\text{Fr}^2 L$; use the same number to size <code>--sponge</code> and <code>--beach_gap</code>

6 Field inventory

Fields the viewer and the slice tools should offer, with the solver attribute that supplies them. Default air masking follows the lesson that $\omega c/U$ normalisation amplifies air-phase noise by $\sim 1600\times$.

Table 3: Field inventory. “mask” = air phase masked by default.

Field	Source	Status	Notes
ϕ	phi	exists	Isosurface at 0.5; thickness histogram (§7)
$\rho(\phi)$	rho	exists	Linear in ϕ
$\tau(\phi)$	tau_fld	exists	Show $\tau - \frac{1}{2} \log$ scale; the light phase is the risk
p^*	p_star	exists	Under WB this <i>is</i> p_d
p_d, Π_h, p	p_star, compute_Pi_h, measure_forces	exists	Three-layer pressure view; rest test $p_d = 0$
$u, u $	ux,uy,uz, get_velocity_magnitude	exists	mask for vorticity-derived quantities
S_{ab}	Sxx..Syz	exists	Stored state; no differencing
Dissipation $2\nu S:S$	from S , tau_fld	add	One kernel-free tensor expression; per-phase sums for the budget
$\nabla \cdot u$	compute_vel_gradients trace	add	Compressibility map; the observable for s_bulk
ω, Q , helicity	kinetic_viz (central differences)	exists	Could instead use S + antisymmetric part from the gradient kernel
$a^{(3)}$ (7), filtered	a3_*, _compute_a4_all	exists	Kinetic layer; magnitude maps
$a^{(4)}$ (6)			
J_ϕ, P_ϕ	Jx_phi..Pyz_phi	exists	Phase-moment layer; rarely needed, keep behind “advanced”
$F_s, F_p, F_\nu, F_b, F_{\text{tilt}}$	compute_all_forces (returns the sum)	add	Return the terms separately behind a flag; mask
$\nabla\phi, \nabla\phi $ vs $4\phi(1-\phi)/\xi$	compute_grad_phi	exists	Limiter-engagement map = where the ratio exceeds 1.6
Solid mask, SDF, cut links	fsi_solid, SDF bundle, faces	exists	Draw the SDF zero level <i>and</i> the voxel contour
MD level map, bands, ghosts	GridLevel topology	exists	Colour by level; show band depth and twin rings
Sponge / beach weights	_build_absorb_field, driver zones	exists	Translucent volumes with strength profile
Safeguard activity	limiter/threshold masks	add	Boolean per cell per step; count + map

7 Monitors: ledgers and budgets before anything else

There is no residual. The Monitors page should open on five health signals, each with a traffic-light threshold and a one-line explanation of what to do when it turns.

Table 4: Health signals and secondary monitors.

Monitor	Source	Status	Healthy / action
<i>Primary (the five)</i>			

Monitor	Source	Status	Healthy / action
ϕ mass ledger	<code>get_phase_mass</code> ; hull driver's "ledger drift"; MD per-level ledger + injections	exists	$ \text{drift} < 10^{-4}$; injections should not grow (positive-feedback failure, Prop. 10.1)
Energy budget residual	live version of <code>h2_energy_budget.py</code> : $W = D_{\text{near}} + D_{\text{far}} + D_{\text{air}} + Z_{\text{beach}} + Z_{\text{floor}} + \Delta KE + \Delta PE + \text{res}$	add	Residual as a bar; large residual = non-conservative operations at the body or sub-grid Stokes layer
Force channel agreement	<code>measure_forces</code> : stress integral vs MEA_ALD	exists	Within a few %; divergence means cut-link / retabulation trouble
Steady-window bracket	driver: window mean [previous window]	exists	Bracket within tolerance = converged in time
WB rest test	$\max p_d $ with $U = 0$, full g	exists	Exactly 0 at rest; any growth is a BC or MD defect
<i>Secondary</i>			
Ma_{inst} , $\max u $ + location	<code>get_max_speed</code> ; hull driver <code>vw_loc</code>	exists	Location feeds the locator (§15)
τ_{min} $\ \nabla \cdot u\ $	<code>tau fld.min()</code> new	exists add	Margin to $\frac{1}{2}$ $\sim \text{Ma}^2$; rises with acoustic ringing; knob <code>s_bulk</code>
Interface thickness histogram	$ \nabla \phi $ at $\phi \in (0.1, 0.9)$	add	Peak at ξ ; a tail = sharpening/limiter fight
Spurious current	$\max u $ in cells far from forcing (Laplace test)	add	Recipe values known from the droplet test
Safeguard counts	limiter/threshold cells per step	add	Non-zero but flat is tolerable; climbing is a warning
KE per phase, surface energy	$\frac{1}{2}\rho u ^2$ masked; $\sigma \times$ area of $\phi = 0.5$	exists/add	Feed the budget
Non-finite guard	driver "non-finite load" check	exists	Auto-stop, auto-checkpoint last good state
MLUPS inst. (windowed) and cum.	drivers (cum.); windowed to add	add	Cum. masks throttling; show both
Achieved GB/s	bytes/node/step $\times$ MLUPS vs device peak	add	The honest efficiency number for a bandwidth-bound code

8 Forces, bodies, attitude

`WBFSI3D.measure_forces` returns C_d , C_l , F_x , F_y , F_z from the stress integral on the cut faces (with $p = \rho c_s^2 p_d + (\Pi_h - \Pi_h^0)$), and `MEA_ALD` from momentum exchange when post-streaming populations are kept. `hull_moving.measure_forces_moments` adds the moment about the CoG. The GUI should show, per body:

- the time series of F_x, F_y, F_z, M_y with the pressure and viscous parts, in the benchmark normalisation (F/mg , M_y/mgL , $C_d = F_x/(\frac{1}{2}\rho U^2 D N_y)$);
- both channels and their difference;
- the steady window with the previous window in brackets, and the averaging length in body lengths (`--avg_L`);
- for floating bodies: the hydrostatic stiffness K (k_{33}, k_{35}, k_{55}) from `hull_hydrostatics.py`, the

held attitude ζ_{eq} , trim, and the quasi-static running attitude $K^{-1}F$ next to the tank reference (`th01_tank_data.py`);

- for forced motion: the fitted added mass and damping (`period_fit.py`, `barge_roll_fit.py`) against the BEM reference (`th01_bem_capytaine.py`) and Vugts;
- for free motion: the 6-DOF state, virtual-mass integrator status, and the retabulation cadence with its MLUPS cost shown next to it.

9 Boundaries and zones

Half the tuning in the hull campaign is zones. Each should be a drawn region with a strength profile, and the GUI should size them from the expected wavelength: `--sponge` (solver sponge relaxing p_d and u toward the stream), `--xbeach` / `--xbeach_strength` (driver beach: $u \rightarrow$ stream, $\phi \rightarrow$ flat, p untouched), `--beach_y`, `--beach_gap`, `--zone_strength`, `--floor_fric`, `--mass_correct` {beach, global, off}, `--pierce_bc`, phase-wall pinning (`phi_top`, `phi_bot`, `pin_phase_walls`), inlet ramp (`--ramp_L`, cubic) and the hull-frame body force $\rho dU/dt$ (`--no_frame_accel` to drop it). In MD the sponge strength is exponent-rescaled per level (D13); show the per-level value.

10 Multidomain

A dedicated view: level map coloured by level; per-level table of $\nu^{(d)}$, $\sigma^{(d)}$, $M^{(d)}$, $g^{(d)}$, $\tau^{(d)}$ with the 2^d scaling shown explicitly and the τ -floor choice flagged as a *physics* choice (it runs level d at $2^d \times \text{Re}$); band depth, overlap, restriction margin, even-wrap on periodic axes, phase-free root on/off; the hopping patch following a moving body (Tier 2); per-level MLUPS and mass ledger; S^{neq} factor mode (`dorschner` / `derived`). Driver flags: `--levels`, `--z1`, `--z2`, `--margin`, `--even_wrap`, `--root_phase_free`, `--sneq_mode`, `--no_tau_floor`, `--mass_fix_every`, `--no_full_f2c`.

11 Run: backends, cost, queue, reproducibility

Backend badge. Metal / CUDA / PyTorch fallback, from the extension import line. Never let a fallback run start without a confirmation.

Cost estimate before launch. From the MLUPS table (M1 Pro: RTI 84.6, Colagrossi 32.5, breaking wave ~ 45 , MD RTI 55.3, Magnus 47.8; per-step retabulation 5.9 / 0.63; rack ~ 1.0 GLUPS on the ported kernels) and the step count: “ ≈ 4.2 h on the M4 Pro, 35 min on gpulab”. Keep a small per-machine MLUPS table that the GUI updates from completed runs.

Queue. Mirror `rack/status.sh`: GPU utilisation, memory, power, temperature, clocks; “card busy with someone else’s job” detection (the rack is shared); running jobs with elapsed time; queued jobs; last log lines; disk free. Jobs are launched as the existing drivers with a generated command line; the GUI tails the log and parses the telemetry (§18). Local MPS and remote CUDA from the same dialog.

Reproducibility. A live “equivalent command line” pane that updates as settings change (the ParaView-trace idea applied to run configuration); the provenance `.npz` as the project record; the `_viz` tag rule (figure runs never overwrite published results, and are checked against the published coefficients before use); warm start (`--init_state`) with the grid check `load_state` already enforces; a JSON sidecar for every exported image (`kinetic_viz` already writes one).

12 Outputs and the timeline

Four kinds, four markers:

1. **Traces** (`--measure_every`): forces, ledgers, $\max|u|$ and location, energies; cheap; feed Monitors.
2. **Slices** (mid-span, as most result archives store): cheap 2-D fields at `--save_every`; feed the slice tools.
3. **Viz snapshots** (`--viz-every`, `--viz-dir`): full 3-D `ux,uy,uz,phi,solid`; expensive (CPU transfer + compression); feed the viewer; one `.npz` per snapshot, atomic rename, so the viewer can watch a directory safely.
4. **Restart checkpoints** (`--save_state`): the full moment state (`STATE_KEYS + a^{(3)}`); large; “warm start from here” on right-click.

The GUI should let the user set cadences for each independently, show the disk cost, and never conflate a viz snapshot with a restart point. Auto-checkpoint the last good state when the non-finite guard trips.

13 Viewer: adopt the kinetic_viz contract

Do not reinvent the viewer; port the layer set and the conventions:

- Layers: **surface** ($\phi = 0.5$ isosurface, adjustable), **obstacle** (mask contour, and the SDF zero level when available), **streamlines** (RK45 tubes, speed-coloured, deterministic seeds in liquid cells), **speed** (planar slice), **vorticity** (RGBA volume: Q colour, $|\omega|^2$ opacity), **q** (positive- Q isosurface), **helicity** (signed plane).
- Colour ranges fixed to the 99th percentile of the first frame and held across frames; explicit `--speed-max` / `--q-max` for cross-run comparison; clipped extremes recorded in the sidecar.
- Air mask *on by default* for vorticity, Q and pressure panels; a toggle.
- Derivatives exclude stencils touching air or solid (one-node halo); say so in the panel rather than hiding it.
- Crop in lattice indices with preserved origin; stride as a preview only.
- MD patches are not auto-stitched; show them with true origin and spacing.
- Export: PNG + JSON sidecar, VTI for ParaView, PNG sequence for FFmpeg.

The frontend’s rendering stack is the colleague’s choice; the *contract* (input shape, origin/spacing, masks, percentile ranges, sidecar) is not.

14 Validation cards and recipes

A new project starts from a validated recipe card, not a blank canvas. Each card carries the literature anchor, the parameter recipe (math note §14), the driver, the reference data file, and a gate. Users perturb from there; the project keeps the lineage (“derived from Colagrossi WB; Fr 0.30 $\rightarrow$ 0.45”) and the GUI flags departures from the validated envelope.

Table 5: Recipe cards to ship.

Card	Recipe (from the math note)	Driver / reference	Gate
RTI (Fakhari)	At = 0.5, Re = 3000, Ca = 0.26, Pe = 1000, $\xi = 5$, $t_0 = 16000$, $s^\phi = (1, 1.5, 1.5, 1.5)$, $\rho_H:\rho_L = 3:1$	<code>run_rti3d_</code> <code>multiphase.py</code>	Spike/bubble fronts vs reference; fixed-Cn ladder
Breaking wave (Banari grid3)	$N_x = 400$, $W/L = 0.25$, $L_z/L = 0.30$, $c_{\text{lat}} = 0.015$, $\xi = 5$, $M = 0.02$, slope 0.08	<code>run_breaking_</code> <code>wave3d_md.py</code>	Jet geometry and energy vs Banari; $c_{\text{lat}} = 0.02$ is a known failure

Card	Recipe	Driver / reference	Gate
Colagrossi WB cylinder	Fr-aware U , $g = (U/\text{Fr})^2/D$, $M = 0.05$ ($\text{Fr} < 0.8$), $\text{Bo} = 200$, sponge $8D$, $s_{xy}^\phi = 1.8$, limiter on	<code>run_cylinder3d_</code> <code>colagrossi.py</code>	C_d , C_l , surface profile vs Colagrossi 2018
Oscillating cylinder (Dütsch)	$\text{KC} = 5$, $\text{Re} = 100$, $U_{\max} = 0.04$, domain $20D$	<code>run_cylinder3d_</code> <code>oscillating.py</code>	Force history vs Dütsch
Couette spin gate	$U_s = 0.04$, $\text{Re}_\Omega = 100$, box $12D$	<code>run_cylinder3d_</code> <code>spinning.py</code>	Analytic torque
Magnus	$\text{Re} = 100$, $U = 0.05$, $30D \times 16D$, spin ramped	<code>run_md_magnus_patch.</code> <code>py</code>	Lift vs literature anchor (to verify)
Sedimentation	$\rho_r = 1.25$, $\text{Ga} = 19.6$	<code>run_cylinder3d_</code> <code>sedimenting.py</code>	Terminal velocity vs reference
Vugts barge (H2-c)	$B/T = 2$, forced heave/roll	<code>run_barge_roll.py</code>	Added mass / damping vs Vugts
TH01 hull	$L = 256$, Fr , Re_{ref} , tank $G/Q/P$ zone presets	<code>run_hull_static.py</code> , <code>run_hull_speed.py</code>	Heave/trim/drag vs SYRF tank; BEM intercepts
Hydrofoil (Parkin)	$\text{Fr} = 0.95$, $h/c = 1.8$, $c = 128$, $\text{Re} = 4000$	<code>run_hydrofoil_</code> <code>parkin.py</code>	Surface profile vs Parkin–Wu

Convergence-ladder builder. Refinement in this code means fixed-Cn ($\xi \propto N$), fine-level mobility, and a steady-window extraction per rung. The GUI generates the ladder from a card, estimates cost per rung, queues it, and assembles the convergence plot and order estimate at the end (`hull_ladder_analysis.py`, `plot_hull_ladder.py`, `make_hull_ladder_tex.py` are the existing pieces).

15 The trouble locator

The hull driver already reports $\max |u - U|$ with its location relative to the CoG and stops on a non-finite load. Make this a feature: on a NaN or a hot spot above threshold, auto-checkpoint the last good state, stop, jump the camera to the cell, drop a marker, and show the local ϕ , τ , safeguard flags, and whether the cell sits in a band, a sponge, or a cut-link shell. This one feature will save more user hours than any rendering option.

16 What sets it apart — ranked

1. **Recipe-first projects with lineage and envelope warnings.** No CFD GUI ships with its default state being a validated run and a gate.
2. **Ledgers and budgets as the health panel.** Mass ledger, energy-budget residual, two-channel force agreement, steady-window bracket, WB rest test.
3. **The feasibility triangle** (N , Ma , τ) with Cn and Pe_ϕ , live, with recipes plotted.
4. **The interface as a first-class object:** thickness histogram, spurious current, limiter and threshold activity maps. Scientific honesty built in.
5. **Strain rate is state:** dissipation, enstrophy, Q exact; and the *kinetic layer* (S^{neq} , $a^{(3)}$, filtered $a^{(4)}$) as a “where is the model working hardest” map that only a moment-based Hermite solver can produce.
6. **The trouble locator.**
7. **The convergence-ladder builder** with cost per rung.
8. **One launcher for the Mac and the rack**, tenant-aware.

9. **Reproducibility by construction:** live CLI line, provenance `.npz`, `_viz` tag rule, figure sidecars.
10. **Four-layer pressure view** with the one-click WB rest test.

17 Build sequence

Phase 0 — structure and units (no solver changes).

New sidebar; unit map with the three-form readouts; dimensionless-first setup form with the checks of §5; backend badge; live CLI line; load existing provenance `.npz` archives and `kinetic_viz` snapshots. Ship the recipe cards as static project templates.

Phase 1 — monitors, forces, viewer (small driver additions).

Telemetry stream (§18) emitted by the drivers; Monitors page with the five health signals; forces with both channels and the bracket; viewer with the `kinetic_viz` layers and the air mask; timeline with four marker kinds; trouble locator; windowed MLUPS.

Phase 2 — validation and ladders.

Reference overlays and gates per card; convergence-ladder builder; cost model; envelope warnings from lineage.

Phase 3 — MD, queue, kinetic layer.

MD view; rack queue mirroring `status.sh`; safeguard activity maps; dissipation and divergence fields; energy-budget closure live; kinetic-layer fields; force-term layers.

18 Telemetry schema (so the GUI never imports torch)

Drivers print a JSON line every `--measure_every` steps (stdout or a `.jsonl` beside the `.npz`); the GUI tails it. Proposed record; fields marked † need a small driver addition, the rest are already computed.

```
{
  "step": 12000, "t_lat": 12000, "t_phys": 0.84, "t_star": 2.31,
  "backend": "metal", "mlups_inst": 61.2, "mlups_cum": 63.0,
  "ma_inst": 0.071, "tau_min": 0.5083,
  "umax": 0.0412, "umax_loc": [212, 4, 96],
  "mass_ledger": {
    "phi": -1.3e-5, "levels": [-1.3e-5, 2.0e-6], "injected": 0.0
  },
  "ke": {
    "water": 3.1e-3, "air": 8.0e-6, "pe_surface": 1.2e-3
  },
  "div_u_norm": 2.4e-4,
  "forces": {
    "Fx": -1.1e-2, "Fy": 0.0, "Fz": 0.42, "My": 3.0e-4,
    "Fx_p": -8.0e-3, "Fz_p": 0.41, "mea_Fx": -1.08e-2
  },
  "window": {
    "Fx": -1.10e-2, "Fx_prev": -1.12e-2
  },
  "safeguards": {
    "limiter_cells": 0, "threshold_cells": 0
  },
  "budget": {
    "W": 0.0, "D_near": 0.0, "D_far": 0.0, "D_air": 0.0,
    "Z_beach": 0.0, "Z_floor": 0.0, "dKE": 0.0, "dPE": 0.0, "res": 0.0
  },
  "snapshot": null, "checkpoint": null, "nonfinite": false
}
```

†: `mlups_inst`, `div_u_norm`, `safeguards`, `budget`, `pe_surface`, per-phase `ke`. Everything else is a reformatting of what the drivers print today. A matching `run_spec.json` (the settings the GUI serialises) should round-trip to the CLI line and be stored inside the provenance `.npz` as a string.

19 Open questions for Colton and the frontend engineer

1. Process model: does the GUI launch the drivers as subprocesses (simplest; keeps the validated CLI path) or embed the solver? The telemetry schema assumes subprocesses and a tailed log; this also makes the rack a first-class target.
2. Rendering stack: port `kinetic_viz` (PyVista/VTK, Python) or reimplement the contract in the GUI's own renderer? If the GUI is web-based, the VTI export is the bridge.

3. Which drivers get the telemetry hook first? Suggest `run_hull_speed.py`, `run_cylinder3d_colagrossi.py`, `run_rti3d_multiphase.py` — they cover WB+FSI, MD, and single domain.
4. Where does the run spec live: inside the `.npz` only, or also a sidecar JSON beside it for grepability?
5. Energy-budget boxes (near/far/air) are case-specific in `h2_energy_budget.py`; the GUI needs a general box definition (body-relative radii) or a per-card default.
6. Unit map ownership: the drivers currently carry physical anchors ad hoc (`F_N`, `heave_mm`); a single `units` block in the run spec would let the GUI convert everything consistently.

A Worked map: `run_hull_speed.py` flags → controls

Table 6: Every flag of `run_hull_speed.py` and where it lives in the proposed architecture.

Flag	Page	Control
<code>--L, --config</code>	Geometry	Body length in cells; patch classification (CB / HKR / HKr / full)
<code>--Fn, --Re_ref, --g</code>	Fluids & Interface	Dimensionless-first form; derived U , ν , τ shown
<code>--xi, --Bo, --mobility, --rho_L</code>	Fluids & Interface	Interface block; σ derived from Bo
<code>--sink_cells, --trim, --no_equilibrate</code>	Bodies	Attitude offsets on top of the equilibrium solve
<code>--pad_up, --pad_down, --pad_side, --depth, --air</code>	Lattice	Tank layout in body lengths; drawn in the 3-D view
<code>--sponge, --xbeach, --xbeach_strength, --beach_y, --beach_gap, --zone_strength, --floor_fric</code>	Boundaries & Zones	Zone regions with strength profiles; sized from $2\pi Fr^2 L$
<code>--mass_correct, --pierce_bc</code>	Boundaries & Zones	Mode selectors with one-line explanations
<code>--travel, --ramp_L, --no_frame_accel, --steps</code>	Run	Duration in body lengths; ramp; frame force toggle; step override
<code>--avg_L, --measure_every, --print_every</code>	Run / Monitors	Averaging window; telemetry cadence
<code>--band_cells, --refine, --sdf_backend, --cpt_path</code>	Geometry	SDF band and backend (libigl / CPT)
<code>--body_on_cpu, --no_gpu_kernels, --device</code>	Run	Backend and placement; fallback confirmation
<code>--init_state, --save_state</code>	Run / timeline	Warm start; restart checkpoint cadence
<code>--tag, --outdir, --smoke</code>	Run	Naming (the <code>_viz</code> rule), output folder, smoke preset